



Data Visualization

LABORATORIO

Capitolo 8 – Alberi



Argomenti del giorno

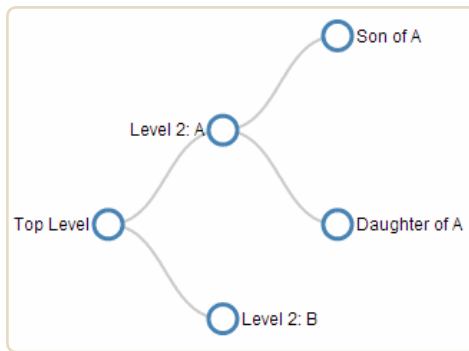
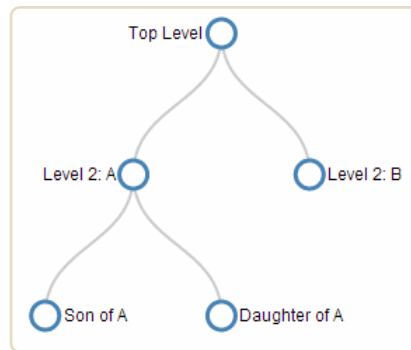
- Alberi
 - Struttura
 - Orientamento
- Animazioni e transizioni

Quando usare gli alberi

Gli alberi sono un tipo **speciale** di grafo e vanno usati quando i dati seguono una **gerarchia** chiara, quindi quando ogni nodo ha esattamente un **genitore** (tranne la radice).

Un albero è un grafo **aciclico**, **connesso** e **orientato** dalla radice verso i figli.

Sono ottimi per **rappresentare strutture gerarchiche**, per esempio file system, tassonomia, organigrammi, etc.



Oggetti annidati

In questo caso il dataset è un oggetto **annidato**: ogni elemento della collezione è composto da due attributi: **name** e **children**.

A sua volta **children** può avere all'interno **altri** elementi con la sua stessa struttura.

Quando i figli, children, non avranno più **discendenti**, allora ci troveremo davanti a un nodo **foglia** dell'albero.

```
{
  "name": "Cinema",
  "children": [
    {
      "name": "Warner Bros",
      "children": [
        {
          "name": "Action",
          "children": [
            { "name": "Superhero" },
            { "name": "Spy" }
          ]
        },
        {
          "name": "Drama",
          "children": [
            { "name": "Historical" },
            { "name": "Courtroom" }
          ]
        }
      ]
    }
  ]
},
```

```
{
  "name": "Disney",
  "children": [
    {
      "name": "Animation",
      "children": [
        { "name": "3D Animation" },
        { "name": "2D Animation" },
        { "name": "Stop Motion" }
      ]
    },
    {
      "name": "Adventure",
      "children": [
        { "name": "Fantasy" },
        { "name": "Family Adventure" }
      ]
    }
  ]
},
```

Struttura gerarchica

Tramite **d3.hierarchy()** trasformiamo un JSON in una struttura gerarchica **navigabile**.

Con **.descendants()** accediamo ai nodi **figli** e con **.links()** agli **archi**. Ogni nodo avrà attributi come **coordinate**, **l'altezza** del suo sotto albero, la **profondità** a cui si trova, etc.
Anche gli archi avranno informazioni analoghe.

Tramite la funzione **d3.tree()** generiamo il **layout** ad albero dandogli le dimensioni.

Successivamente lo applichiamo alla gerarchia.

```
const root = d3.hierarchy(data);  
console.log("DESCENDANTS: ", root.descendants());  
console.log("LINKS: ", root.links());  
  
const treeLayout :... = d3.tree().size([width , height]);  
treeLayout(root);
```

Creazione dei cerchi animati

Creiamo un gruppo che conterrà tutti i nodi e poi, per ogni nodo, creiamo un gruppo che trasliamo in base alla posizione che avrà il nodo. Ogni gruppo poi avrà come classe nodes.

Per ogni gruppo/nodo aggiungiamo un cerchio con i soliti attributi stilistici. Vogliamo far apparire il nodo con una transizione. Definiamo quindi prima la transizione e **solo successivamente** il raggio del cerchio.

In questo modo i cerchi appariranno in 0.8 secondi.

```
const nodesGroup = gContainer.append('g').attr('class', 'nodes');  
//creiamo un gruppo per ogni nodo, posizionandolo in base alle coordinate calcolate dal layout  
const node = nodesGroup.selectAll('.node')  
  .data(root.descendants())  
  .join('g')  
  .attr('class', 'node')  
  .attr('transform', d => `translate(${d.x}, ${d.y})`);  
  
// Per ogni nodo, aggiungiamo un cerchio e del testo  
// Iniziamo con un cerchio di raggio 0 e poi facciamo una transizione per farlo crescere  
node.append('circle')  
  .attr('r', 0)  
  .attr('fill', '#69b3a2')  
  .attr('stroke', '#333')  
  .attr('stroke-width', 2)  
  .transition()  
  .duration(800) //ms  
  .attr('r', 8);
```

Aggiunta delle etichette

Per ogni nodo aggiungiamo un testo vicino che ci indichi il nome.

Creiamo una **funzione** che posizioni il testo sopra al cerchio se si tratta di un nodo interno, mentre se si tratta di un nodo foglia, allora deve alternare tra sopra e sotto, così da evitare sovrapposizioni.

Specifichiamo che se il nodo è un padre, allora il testo sarà in grassetto.

```
// Aggiungiamo il testo, posizionandolo sopra o sotto il nodo a seconda se è una foglia o meno
node.append('text')
  .text(d => d.data.name)
  .attr('dy', (d, i) =>{
    if(!d.children)
      return i % 2 === 0? '-12px' : '20px'; // Se è una foglia, alterna sopra e sotto
    return '-12px'; // Se non è un nodo foglia, posiziona il testo sopra
  })
  .attr('text-anchor', 'middle')
  .attr('fill', '#333')
  // user-select: none per evitare che il testo venga evidenziato quando si clicca
  .style('user-select', 'none')
  // Se il nodo ha figli, rendi il testo in grassetto
  .style('font-weight', d => d.children? 'bold' : 'normal');
```

Creazione degli archi

Anche in questo caso, creiamo un gruppo che conterrà tutti gli archi e gli specifichiamo la classe link e che deve essere posizionato come primo figlio dell'SVG container. Questo perché così viene «disegnato» prima l'arco e sopra il nodo.

Creiamo gli archi che collegano i nodi tramite la funzione **d3.linkVertical()**, utile quando vogliamo creare degli alberi orientati dal **verticalmente**.

(Tramite **.linkHorizontal()** invece si possono creare archi appositi per alberi **"orizzontali"**.)

Queste funzioni prendono in input le coordinate x e y degli archi definite tramite la funzione **d3.hierarchy()**

```
const linkGroup = gContainer.insert('g', ':first-child').attr('class', 'links');

// Per ogni link, aggiungiamo un path che collega il nodo padre al nodo figlio
const links = linkGroup.selectAll('.link')
  .data(root.links())
  .join('path')
  .attr('class', 'link')
  .attr('d', d3.linkVertical() //crea un path verticale che collega i nodi padre e figlio
    .x(d => d.x)
    .y(d => d.y))
  .attr('fill', 'none')
  .attr('stroke-width', 2)
  .attr('stroke', '#ccc')
  .attr('stroke-dasharray', function() {
    // this.getTotalLength() restituisce la lunghezza totale del percorso del link
    const totalLength = this.getTotalLength();
    return totalLength + " " + totalLength;
  })
  // Impostiamo l'offset iniziale pari alla lunghezza totale del percorso,
  // in modo che il link sia completamente nascosto
  .attr('stroke-dashoffset', function() {
    return this.getTotalLength();
  })
  .transition()
  .duration(1000)
  // Dopo la transizione, impostiamo l'offset a 0, facendo apparire il link come se fosse disegnato
  .attr('stroke-dashoffset', 0);
```


Animazione degli archi

Come per i nodi, **animiamo** gli archi: in questo caso definiamo delle **funzioni** che vanno a lavorare sugli attributi **stroke-dasharray** e **stroke-dashoffset**.

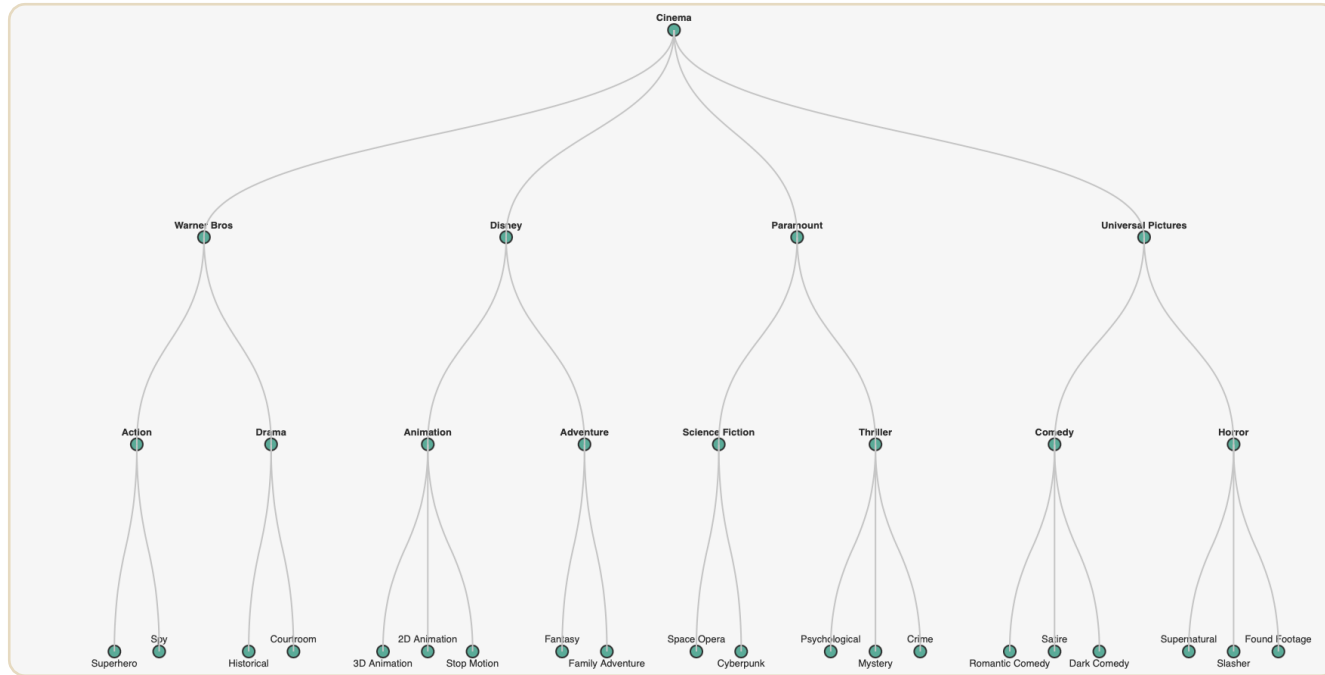
Sfruttiamo la lunghezza degli archi per creare degli effetti visivi.

Alla fine aggiungiamo la **transizione** per riportare a zero l'offset e vedere gli archi normalmente.

```
const linkGroup = gContainer.insert('g', ':first-child').attr('class', 'links');

// Per ogni link, aggiungiamo un path che collega il nodo padre al nodo figlio
const links = linkGroup.selectAll('.link')
  .data(root.links())
  .join('path')
  .attr('class', 'link')
  .attr('d', d3.linkVertical() //crea un path verticale che collega i nodi padre e figlio
    .x(d => d.x)
    .y(d => d.y))
  .attr('fill', 'none')
  .attr('stroke-width', 2)
  .attr('stroke', '#ccc')
  .attr('stroke-dasharray', function() {
    // this.getTotalLength() restituisce la lunghezza totale del percorso del link
    const totalLength = this.getTotalLength();
    return totalLength + " " + totalLength;
  })
  // Impostiamo l'offset iniziale pari alla lunghezza totale del percorso,
  // in modo che il link sia completamente nascosto
  .attr('stroke-dashoffset', function() {
    return this.getTotalLength();
  })
  .transition()
  .duration(1000)
  // Dopo la transizione, impostiamo l'offset a 0, facendo apparire il link come se fosse disegnato
  .attr('stroke-dashoffset', 0);
```

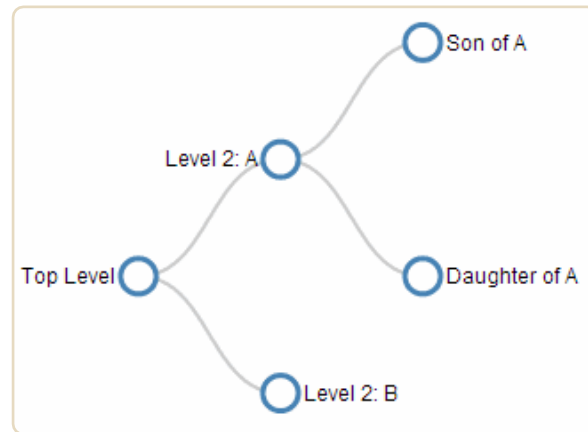
Risultato



Esercizio 1: orientamento del layout

Partendo dall'albero a sviluppo verticale costruito durante la lezione:

- Trasformare l'orientamento dell'intera visualizzazione da verticale a orizzontale.
- Assicurarsi che gli archi (links) vengano generati correttamente per il nuovo orientamento.
- Riposizionare i nodi e le relative etichette di testo in modo coerente con la nuova disposizione.





Esercizio 2: interattività e hovering

Implementare un effetto di hovering sui nodi: al passaggio del mouse, il cerchio deve cambiare colore e aumentare lo spessore del bordo.

Al "mouseout", il nodo deve tornare istantaneamente (o con una breve transizione) allo stato stilistico originale.





Esercizio 3: animazione al click

Aggiungere un listener per l'evento "click" su ogni nodo.

Al click, il raggio del cerchio deve aumentare gradualmente tramite una transizione.

Configurare il testo dell'etichetta affinché si sposti automaticamente per non essere sovrapposto dal cerchio ingrandito.

